

When the Next Step Is Not One Step: Distribution-Aware Execution Modeling for Concurrent Go Programs

Kaviru Hapuarachchi

University of Colombo School of Computing,
35 Reid Avenue, Colombo 07, Sri Lanka
`2022is031@stu.ucsc.cmb.ac.lk`

Abstract. Training a model to predict the next step in a concurrent program is harder than it looks: two runs of the same program from the same trace prefix can produce different next events, both valid, because the scheduler is nondeterministic. A model trained against a single label is learning to guess one outcome of a random process. We turn this around and use the nondeterminism as a training signal. We run each program many times, aggregate the observed next events into an empirical distribution, and fine-tune a 7B model to match that distribution with a KL objective. On 798 held-out predictions drawn from real production Go bugs (CockroachDB, Kubernetes, gRPC, etcd), fine-tuning on fewer than a thousand traces reaches 36.2% accuracy, ahead of Gemini 3.5 Flash used zero-shot (34.8%) and the same model without fine-tuning (28.6%). Distribution training matches cross-entropy on accuracy (35.8% vs. 36.2%) while reducing Expected Calibration Error from 0.205 to 0.169. We also derive a formal goroutine-leak signature for a class of select-blocked goroutines where $P(\text{GoUnblock}) = 0$ holds by scheduler semantics, not by learning. We release the dataset, trained adapters, and all tooling.

Keywords: concurrent execution modeling; nondeterminism; distribution estimation; model calibration; Go; goroutine scheduling; concurrency bugs

1 Introduction

A model trained on execution traces learns to predict what a program does, not just what it looks like. This is the idea behind Code World Models (CWMs): trained on runtime state snapshots after each statement, a language model learns the state transition function, which improves verification, debugging, and code generation [1]. For sequential Python the approach works well.

But it rests on a quiet assumption: that execution is deterministic. Given a state and the next statement, there is one next state. For concurrent code that assumption fails. When goroutines run in parallel and communicate over channels, the scheduler interleaves them differently on every run. The same prefix

can be followed by a block, a start, or an unblock, all equally valid. Train a model to predict a single next event here and you are asking it to memorize one arbitrary outcome of a random process. It will learn very little.

This is not a minor technicality. Concurrency bugs, deadlocks, data races, goroutine leaks, have no sequential analogue. They depend on schedule, evade unit tests, and are hard even for experts to anticipate [3, 4]. If execution-trace models are to be useful where the bugs are hardest, they must handle the nondeterminism that defines the setting.

We propose treating the nondeterminism as a signal rather than noise. Run a concurrent program many times and the distribution of next events is not random error. It is a measurement of which futures the scheduler actually permits. We reformulate next-event prediction as distribution estimation: the target for a prefix is the empirical distribution over next events observed across repeated runs, and we train a model to match it (Fig. 1).

We make the following contributions:

- We show that a 7B model fine-tuned on fewer than a thousand concurrent traces reaches 36.2% next-event accuracy on held-out production bugs from CockroachDB, Kubernetes, gRPC, and etcd, beating Gemini 3.5 Flash zero-shot (34.8%) and the same model without fine-tuning (28.6%).
- We show that training with a KL objective against empirical distributions matches cross-entropy on accuracy (35.8% vs. 36.2%) while improving calibration: Expected Calibration Error drops from 0.205 to 0.169 and model entropy correlates with program nondeterminism.
- We derive a formal leak signature for select-blocked goroutines, showing that $P(\text{GoUnblock}) = 0$ at every trace depth follows from Go scheduler semantics, not from learning.
- We report where the approach falls short: accuracy plateaus near 35–36% regardless of model or objective, rare event types are never learned, and multi-step predictions lose scheduler coherence after roughly one step.

We are not claiming a deployable bug detector or a production-ready execution simulator. The contribution is a formulation, a dataset, and a set of baselines that show nondeterminism-aware training works and make clear what would need to change for it to work better.

2 Background and Related Work

Code world models and execution traces. A world model learns an environment’s transition function: state and action in, next state out [1]. CWMs apply this to program execution, training on action-state pairs from interpreter traces so the model predicts the next action and resulting state from a partial trace [1]. Meta’s 32B CWM, trained on Python traces and agentic trajectories, showed that this grounding improves coding and reasoning [1]. A follow-up study of CWM failure modes found that errors concentrate in two regimes: token-budget exhaustion on long traces and string-valued state confused by subword tokenization [2]. Both studies assume deterministic, sequential execution.

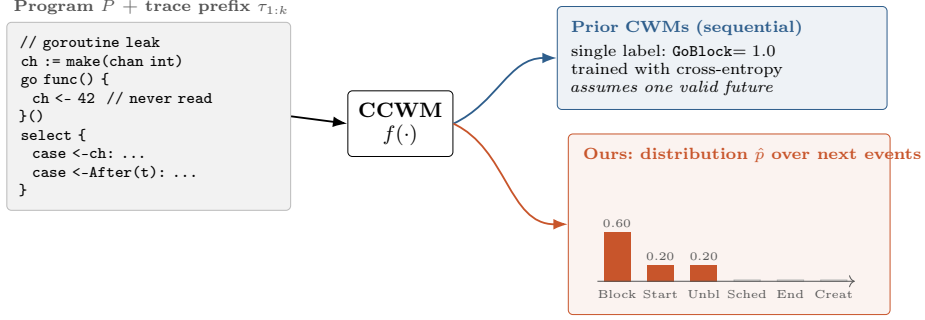


Fig. 1. Concurrent execution has many valid next steps, not one. Prior Code World Models (blue) predict a single next event and train against it with cross-entropy, which works for sequential code but is ill-defined when the same prefix can legitimately produce several different events. We predict a full distribution over next events (orange), with targets derived from repeated runs of the same program. The right panel shows an example distribution for a goroutine-leak program: GoBlock dominates and $P(\text{GoUnblock}) = 0$, a signature that follows from the scheduler, not from learning.

Concurrency and LLMs. Concurrent code introduces nondeterministic scheduling and bug classes, races, deadlocks, starvation, that have no sequential equivalent. Unit-test evaluation cannot systematically explore thread schedules [3]. The CONCUR benchmark targets concurrent code generation, judged by model checking [3]. Our problem is different: modeling the execution of concurrent programs as a learned transition function, using scheduling nondeterminism as a distributional target. The GoKer/GoBench corpus of real Go concurrency bugs [4] provides our held-out test programs.

Distribution matching and calibration. Training against a target distribution rather than a point label is an established technique: soft-target objectives improve language-model calibration, often using an empirical distribution from a few hundred samples as the target [5, 6]. Our contribution is not distribution matching itself but its source. The nondeterminism of concurrent execution, observed through repeated runs, gives a principled empirical target over next events. We are not aware of prior work deriving distributional training targets from concurrent execution traces.

Table 1 positions our work against the two closest lines of prior research.

3 Problem Formulation

We consider Go programs that spawn multiple goroutines. The runtime tracer emits scheduler events and we work with six event types covering goroutine lifecycle and synchronization:

$$\mathcal{E} = \{\text{GoBlock}, \text{GoCreate}, \text{GoEnd}, \text{GoSched}, \text{GoStart}, \text{GoUnblock}\}.$$

Table 1. Comparison against prior execution-trace modeling and the closest concurrency benchmark across dimensions relevant to this problem. This work is the first to combine execution-trace modeling with concurrent programs and nondeterministic distributional targets.

	Sequential CWM CONCUR This work		
	[1]	[3]	
Concurrent programs	no	yes	yes
Models execution (vs. gen.)	yes	no	yes
Nondeterministic targets	no	n/a	yes
Real-world bug test set	no	no	yes (GoKer)
Calibration evaluated	no	no	yes

A trace is a sequence of snapshots $\tau = (s_1, \dots, s_n)$, each recording the triggering event type, the goroutine id, and per-goroutine status (running / runnable / blocked / dead). Given a prefix $\tau_{1:k}$ and program source P , the task is to predict the event type of s_{k+1} .

From label to distribution. Standard CWM training takes the next event as one label and minimizes cross-entropy. Because execution is nondeterministic, repeated runs of P from comparable prefixes produce different valid next events. For a group g of runs sharing a program and prefix depth, let $c_g(e)$ count next events of type e . The empirical distribution and its Dirichlet posterior (Jeffreys prior $\alpha=0.5$) are

$$\hat{p}_g(e) = \frac{c_g(e)}{\sum_{e'} c_g(e')}, \quad \tilde{\alpha}_g(e) = \alpha + c_g(e). \quad (1)$$

The learning target is $\hat{p}_g$, not a single label. We approximate “same prefix” by matching split depths (25/50/75% of the trace). Because interleavings differ across runs, grouped prefixes are structurally comparable but not byte-identical. We revisit this approximation in Section 9.

4 Dataset and Evaluation Setup

Programs. We assembled 130 concurrent Go programs in three groups (Table 2). Hand-crafted programs cover channel, mutex, select, pipeline, waitgroup, and fan-in/out patterns, with intentional deadlocks, races, and leaks. Generated programs are synthesized with randomized parameters and optional injected bugs, each verified to compile. Real-world programs are reduced concurrency-bug kernels from the GoKer/GoBench corpus [4], drawn from production systems including CockroachDB, Kubernetes, gRPC, etcd, Istio, and Moby. Each program carries metadata describing outcome, pattern, goroutine count, and expected nondeterminism.

Traces. Each program is compiled with the race detector and run five times under the runtime tracer. Differing interleavings across runs are the variability

Table 2. Program corpus. All 66 real-world GoKer programs are held out from training, so accuracy on those programs measures out-of-distribution generalization to code the model never saw during fine-tuning.

Group	Count	Source
Hand-crafted	26	All patterns; intentional bugs
Generated	38	Parameterized synthesis with injected bugs
Real-world	66	GoKer/GoBench kernels (held out for test)
Total	130	

we exploit. From each run we form examples at 25/50/75% prefix depth. Dead-locking programs emit a no-next-event marker and are excluded from distribution aggregation. The tracer exposes scheduler events but not channel buffers, mutex holders, or local variables, so the model must reason from scheduling behavior and program structure.

Splits. We hold out all 66 real-world programs for evaluation and train only on hand-crafted and generated programs. This yields 945 training examples and 798 held-out next-event predictions, plus 75 aggregated empirical-distribution groups.

5 Method

Format. Each example renders the program source, the partial trace as JSON, and the current goroutine states, then asks for the next event. The point target is a JSON object `{"event_type":..., "goroutine_id":...}`; the distribution target is the six-way vector $\hat{p}_g$. Prompts are left-truncated at the source so the target is never cut off.

Cross-entropy baseline. We fine-tune Qwen2.5-Coder (1.5B and 7B) with cross-entropy over response tokens using 4-bit QLoRA (rank 16, $\alpha=32$, gradient checkpointing). To approximate the empirical distribution under a point loss, examples are duplicated in proportion to observed next-event frequency.

KL distribution loss. The distribution objective adds a KL term at the token position that discriminates between event types. With logits z restricted to the six event-type tokens, $q = \text{softmax}(z)$, and empirical target $\hat{p}_g$,

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \text{KL}(\hat{p}_g \parallel q), \quad (2)$$

with $\lambda=0$ recovering the CE ablation. Because all six event types share the leading subword “Go,” we place the KL term at the second, discriminating token. The 7B QLoRA configuration is identical to the CE baseline so that any difference is attributable to the objective alone.

Multi-step coherence probe. To measure how far single-step prediction extends before it breaks, we feed the model’s own predictions back as input: from a real prefix, predict the next event, append it, and repeat for up to 15 steps. A

Table 3. Next-event accuracy on the held-out real-world GoKer set. Fine-tuning on concurrent traces from hand-crafted programs generalizes to real production bugs, outperforming both the zero-shot baseline and Gemini 3.5 Flash. Distribution training with KL loss achieves comparable accuracy to cross-entropy while improving calibration (Section 7).

Model	Accuracy
Qwen2.5-Coder-7B, zero-shot	28.6%
Gemini 3.5 Flash, with thinking	34.8%
Gemini 3.5 Flash, no thinking	35.2%
Qwen2.5-Coder-7B, fine-tuned KL (ours)	35.8%
Qwen2.5-Coder-7B, fine-tuned CE (ours)	36.2%

Table 4. In-distribution accuracy on hand-crafted programs, included for context. These numbers are not directly comparable to Table 3 because the model used here is an earlier Gemini generation evaluated with a different prompt format.

Model	Accuracy
Qwen2.5-Coder-1.5B, zero-shot	29.8%
Gemini (earlier gen., different prompt)	56.0%
Qwen2.5-Coder-1.5B, fine-tuned CE (ours)	40.2%

symbolic scheduler finite-state machine checks each predicted transition against Go invariants (for example, a blocked goroutine cannot be started). We report survival steps, meaning valid transitions before the first violation, along with the per-step event distribution and entropy. We use this as a diagnostic tool in Section 7 to locate the boundary between what the model can and cannot do.

6 Results

Fine-tuning generalizes out-of-distribution. Table 3 shows the main result. Trained on 945 hand-crafted and generated traces, a 7B model reaches 36.2% on real-world GoKer bugs it never saw during training. This beats the same model zero-shot (28.6%) and Gemini 3.5 Flash used zero-shot (34.8–35.2%). Small-scale concurrent-trace supervision transfers to structurally different real code.

Distribution training matches accuracy. KL training reaches 35.8%, which is not meaningfully different from CE’s 36.2%. We view this as the expected outcome: distribution training should not sacrifice accuracy, and the benefit appears in calibration rather than top-1 prediction (Section 7).

An accuracy ceiling near 35–36%. Three approaches, CE fine-tuning, KL fine-tuning, and a strong zero-shot model, all land within one percentage point of each other. That clustering is itself informative: predicting the next scheduler event of a real concurrency bug from traces alone is hard, and neither scale nor objective moves the ceiling. Section 7 traces this to rare events and distribution shift.

Table 5. Per-event accuracy of the KL model on GoKer, alongside train and test frequency. Two events with near-zero training frequency, `GoEnd` and `GoSched`, are never predicted correctly. Meanwhile `GoCreate` achieves 44% accuracy from only 1.5% of training examples, suggesting the model reasons from program structure rather than label frequency.

Event	Train freq.	Test freq.	Accuracy
<code>GoStart</code>	42%	35%	47%
<code>GoBlock</code>	37%	26%	36%
<code>GoUnblock</code>	17%	6%	8%
<code>GoCreate</code>	1.5%	21%	44%
<code>GoEnd</code>	2%	4%	0%
<code>GoSched</code>	0.4%	7%	0%

Reasoning does not help. Enabling thinking in Gemini 3.5 Flash slightly lowers accuracy (34.8% vs. 35.2%). The relevant signal here is structural, not multi-step deductive.

7 Analysis: Where the Ceiling Comes From

Per-event accuracy. Table 5 breaks accuracy down by event type. The model learns common lifecycle events, `GoStart` at 47%, `GoCreate` at 44%, `GoBlock` at 36%, but never predicts `GoEnd` or `GoSched` (both 0%) and rarely gets `GoUnblock` right (8%).

Two effects compound to produce the ceiling. First, class imbalance: `GoEnd` (2%) and `GoSched` (0.4%) are so rare in training that the model never emits them, and KL does not fix this because the empirical targets are themselves skewed toward common events. Second, distribution shift: real GoKer programs exercise `GoCreate` (21% of test) and `GoSched` (7%) far more than the hand-crafted training set does. The `GoCreate` result is worth noting: 44% accuracy from only 1.5% of training examples means the model is picking up structural cues from the program text, not just frequency patterns.

Calibration and uncertainty. Beyond top-1 accuracy, distribution framing improves how well predicted probabilities track reality. In a prompting study over the aggregated groups, predicting a distribution with a reasoning budget lowers Expected Calibration Error from 0.205 (point-prediction baseline) to 0.169, and model entropy correlates with program nondeterminism (Spearman $\rho = 0.412$, $p = 0.007$). In the multi-step probe, the KL model assigns higher entropy to harder programs: 0.945 bits on leak programs vs. 0.773 on race programs. This is uncertainty the cross-entropy model cannot express by design.

The single-step boundary. Table 6 summarizes the 15-step coherence probe on 54 GoKer programs. The first predicted step reflects real execution semantics: on leak programs, predictions are `GoBlock`-dominated with $P(\text{GoUnblock})$ near zero, matching the signature described in Section 8. However, mean survival is roughly one step before the model produces a scheduler-invalid transition. This

Table 6. Multi-step coherence probe on GoKer (15 steps, 3 samples per program). The first predicted step reflects the leak signature, but mean survival drops to roughly one step before the model produces invalid transitions, confirming that single-step training does not transfer to multi-step simulation.

	Outcome	n	Dominant event	Survival	Entropy
Leak		37	GoBlock (50%)	1.11 steps	0.945 bits
Race		17	GoBlock (65%)	0.67 steps	0.773 bits

was expected: the model was trained only on single steps, so using its predictions autoregressively exposes the boundary quickly. The probe is useful because it locates that boundary precisely rather than leaving it as an open question.

8 The Select-Block Leak Signature

One distributional pattern in our data has a formal explanation rather than a statistical one. In a subclass of goroutine leaks, we consistently observe $P(\text{GoUnblock}) = 0$ at every trace depth across all five runs of those programs.

Consider a goroutine G that enters a `select` statement at time t , where none of the case conditions can ever be satisfied by the remaining execution. We call this a *select-block leak*. For such G , a `GoUnblock` event is impossible for all $t' > t$, and so $P(\text{GoUnblock}) = 0$ in the empirical next-event distribution for any split taken after G ’s first `GoBlock`.

The reason is straightforward. In Go, `GoUnblock` fires only when another goroutine sends to or closes a channel that G is waiting on, or releases a mutex that G is waiting to acquire. If no reachable goroutine in the remaining execution can do either, then `GoUnblock` never fires. This is a consequence of the scheduler’s semantics, not something the model learns. It holds regardless of how many `select` cases G has; we verified this for both two-case and four-case selects.

The signature is mechanism-specific and we are careful not to overstate it. Only leaks where the goroutine reaches a permanently blocked select *before* any `GoUnblock` event appears in the trace show the all-zero pattern. Leaks that do legitimate work first, for example receiving items from a channel before blocking, show $P(\text{GoUnblock}) > 0$ at early split depths. We tried using this signal as an unsupervised anomaly detector and found it separates buggy from clean programs only weakly at our scale (Cohen’s $d = 0.29$, not significant at $n=9$). We present it as a precise characterization of one leak class, not as a general-purpose detector.

9 Threats to Validity

Our grouping of runs by split depth is an approximation. We treat runs at the same depth as sharing a prefix family, but because interleavings differ, the actual prefixes are structurally comparable rather than identical. This can blur the

empirical distribution targets, particularly for programs with high nondeterminism where many interleavings are equally likely.

The runtime tracer does not expose channel buffer occupancy, mutex ownership, or local variable values. The model reasons from a partial view of program state, which bounds achievable accuracy and explains some of the confusion in event types like `GoUnblock` that depend on synchronization state the model cannot see.

Several of our secondary analyses rest on small samples. The calibration correlation ($\rho = 0.412$, $p = 0.007$) is statistically significant, but the anomaly detection result ($n=9$, Cohen’s $d = 0.29$) is not. We report effect sizes throughout rather than drawing conclusions from isolated p -values.

The in-distribution Gemini result in Table 4 used an earlier model generation and a different prompt format. We include it to show the relative improvement from fine-tuning, but we do not compare it numerically to the out-of-distribution results in Table 3.

10 Conclusion

We reformulated next-event prediction for concurrent programs as distribution estimation over empirical nondeterministic targets. A 7B model fine-tuned on fewer than a thousand Go traces generalizes to real production concurrency bugs better than a strong zero-shot large model, and distribution training with a KL objective matches cross-entropy accuracy while improving calibration. The 35–36% accuracy ceiling traces to rare-event failure and train-test distribution shift, and multi-step predictions lose coherence after roughly one step because the model was never trained on trajectories.

Three directions follow directly from these findings. Training on trajectories rather than individual steps is the most direct path to extending multi-step coherence. Encoding channel and mutex state in the trace representation would let the model reason about synchronization, not just goroutine lifecycle events. Rebalancing the training set toward the event mix of real Go code would address the class-imbalance ceiling that currently prevents the model from learning rare but important event types. We release the dataset, the cross-entropy and KL adapters, and all tooling to support this line of work.

Appendix: Artifacts and Compute

All artifacts are publicly released. Code, tooling, and scripts are available at [7]. The benchmark dataset is released as [8]. The cross-entropy and KL fine-tuned 7B adapters are released as [9] and [10] respectively; both are 4-bit QLoRA adapters runnable on a single 20 GB GPU.

Total compute cost was \$12 for fine-tuning on a RunPod RTX 4000 Ada and \$60 for Gemini API inference across all zero-shot evaluations.

References

1. FAIR CodeGen Team: CWM: An Open-Weights LLM for Research on Code Generation with World Models. arXiv:2510.02387 (2025)
2. Rahmani, B.: Debugging Code World Models. arXiv:2602.07672 (2026)
3. Huang, J., Mahmud, T., Păsăreanu, C., Yang, G.: CONCUR: Benchmarking LLMs for Concurrent Code Generation. arXiv:2603.03683 (2026)
4. Tu, T., Liu, X., Song, L., Zhang, Y.: Understanding Real-World Concurrency Bugs in Go. In: ASPLOS (2019)
5. Zhang, Y., Schwarzschild, A., Carlini, N., Kolter, Z., Ippolito, D.: Forcing Diffuse Distributions out of Language Models. arXiv:2404.10859 (2024)
6. Baldelli, D., Kuriakose, S., Hashemzadeh, M., Zouaq, A., Chandar, S.: Probabilistic Calibration Is a Trainable Capability in Language Models. arXiv:2605.11845 (2026)
7. Hapuarachchi, K.: Weave: Concurrent Code World Models – Code, Tooling, and Evaluation Scripts. <https://github.com/kaviru2/weave> (2026)
8. Hapuarachchi, K.: weave-bench: Concurrent Go Execution Trace Benchmark Dataset. HuggingFace Datasets. <https://huggingface.co/datasets/kavirubc/weave-bench> (2026)
9. Hapuarachchi, K.: weave-ccwm-qwen2.5-coder-7b-lora: Cross-Entropy Fine-Tuned Adapter. HuggingFace. <https://huggingface.co/kavirubc/weave-ccwm-qwen2.5-coder-7b-lora> (2026)
10. Hapuarachchi, K.: weave-ccwm-qwen2.5-coder-7b-kl-lora: KL Distribution Fine-Tuned Adapter. HuggingFace. <https://huggingface.co/kavirubc/weave-ccwm-qwen2.5-coder-7b-kl-lora> (2026)